PrintBK System Documentation

Self-Service Student Print Web Application & End-to-End Seller Integration

System Deployment & Health Status

Web Client URL: https://printbk-60074625517.development.catalystserverless.in/app/index.html
API Function: https://printbk-60074625517.development.catalystserverless.in/server/api/
Cloud Platform: Zoho Catalyst Serverless (Org ID: 60074625517, Project: printbk)
Status: 100% Deployed, Verified Active with 0 TypeScript/Runtime Errors
Target Counter: HKBK Student Point (Autonomous Self-Service Campus Print Counter)

1. Executive Overview

PrintBK is a high-speed, zero-cost, self-service print web application designed for campus print shops. Students scan a counter QR code on their smartphone, select files (PDF, DOCX, PPTX, or Photos), customize settings in real-time, view live slide-by-slide visual previews, and submit orders with a secure 4-digit pickup code. The shop counter computer ingests incoming jobs via serverless API endpoints, connects to the local printers, and updates job statuses from processing to ready and completed.

2. Core Issues Solved & Final User-Side Capabilities

Feature Requirement	Previous Limitation	Final Production Implementation
Visual PPTX Preview	Missing or extracted plain text	@aiden0z/pptx-renderer vector slide DOM rendering
Accurate Slide Count	PPTX stuck at 1 page count	Multi-tier OpenXML & CFB OLE2 parser detects all 9+ slides
Upload State Hygiene	Flashed empty basket / blank screen	Instant switch in filePicker.ts to animated processing state
Print File Quality	Risk of lossy canvas compression	100% untouched binary buffer cache (rawFileMap) & upload
Price Breakdown UI	Text overlapping, missing math	Expanded cards with explicit math formula (rate x p x copies)
Button Layout Hygiene	Duplicate floating [x] buttons	Suppressed top floating [x]; single red bottom button + delete
Pickup Verification	Unsaved code on server	Persistent 4-digit code in orders table, tickets & receipts

3. Technical Architecture & Document Processing Pipeline

3.1 Full-Fidelity Visual PPTX Renderer

Prior versions utilized plain text regex extractions, losing slide layouts, backgrounds, font styles, and shapes. The application now embeds @aiden0z/pptx-renderer to provide a true presentation preview:

- **Safe DOM Initialization:** The container element (pptxContainerRef) is set to display: block prior to open() so the renderer accurately measures container width, avoiding zero-scale rendering bugs.
- **Interactive Slider:** The Prev and Next buttons trigger viewer.goToSlide(page - 1) seamlessly while updating the active slide indicator (Slide X of Y).
- **Monochrome Filter Toggle:**When students toggle B&W mode, the CSS filter grayscale(100%) contrast(140%) brightness(102%) is applied to the live presentation DOM.
- **Dynamic Slide Count Synchronization:** If PptxViewer detects additional slides during presentation loading, it dispatches an update to the Zustand store, ensuring the price immediately accounts for all slides.

3.2 Multi-Strategy Slide & Page Counter Engine

In student/src/lib/pdfPages.ts, a multi-tier parsing engine inspects documents entirely inside the browser without requiring heavy server round-trips:

File Format	Primary Parsing Strategy	Secondary Fallback Strategy
.pptx (OpenXML)	Counts presentationml.slide+xml in [Content_Types].xml	Inspects <p:sldId> in presentation.xml & normalized paths
.ppt (Binary OLE2)	CFB library reads PIDDSI_SLIDES in DocSummary stream	Scans RT_SlidePersistAtom (0x03F8) in PowerPoint Doc
.docx (Word)	Reads <Pages>N</Pages> from docProps/app.xml	Clusters paragraph tags in word/document.xml
.pdf (Portable Doc)	pdf-lib loads buffer with ignoreEncryption: true	pdfjs-dist worker fallback parsing

3.3 100% Original Upload Quality Assurance

PrintBK enforces strict document integrity. When files are dropped or selected, the original browser File object is mapped in memory into rawFileMap. During cloud upload, FormData sends the untouched binary buffer directly to the Catalyst File Store. No downsampling, recompression, or JPEG conversion takes place.

3.4 Instant Processing State & Life Cycle

In filePicker.ts, fileInputRef.onchange immediately executes useBasket.getState().setProcessing(true) and navigates to the basket screen before parsing starts. An animated "Processing your documents..." overlay displays, eliminating the blank screen or temporary empty basket flash.

4. Pricing Engine, UI Hygiene & Checkout Flow

4.1 Authoritative Pricing Structure

PrintBK implements a dual-layer pricing engine (packages/shared/src/pricing.ts on frontend and functions/api/index.js on backend). The backend recalculates every itemized cost to prevent client-side price tampering:

Print / Service Type	Standard Rate	Billing Metric	Rules & Discounts
B&W Single-Sided	Rs 1.50	Per printed sheet	Standard monochrome printing
B&W Double-Sided (Duplex)	Rs 0.85	Per physical page	Economical paper-saving rate
Colour (Single or Double)	Rs 8.00	Per page	Full vibrancy laser print
Bulk Tier Discount	Rs 0.80	Per page (B&W)	Automatically applied when >= 100 sheets
Spiral Binding Add-on	Rs 30.00	Per book	Clear transparent front + opaque black back
Hard Cover Project Binding	Rs 150.00	Per book	Golden embossed lettering cover
Lamination (A4 / ID)	Rs 25 / Rs 10	Per document	125-micron thermal seal

4.2 UI Cleanliness & Button Hierarchy Fixes

All floating modal sheets (Breakdown, File Settings, Services, Receipt) now utilize noTopClose on SheetShell. The redundant duplicate [x] buttons in the top-right corner have been removed. Every drawer features a clean, single red checkout-style close button ([x Close Settings] / [x Close Price Breakdown]) and an isolated red [Trash Delete] button on the file card to eliminate accidental taps.

4.3 Checkout, UPI QR & 4-Digit Pickup Code Flow

- **Unique 4-Digit Pickup Code:** A secure 4-digit code (e.g. 4821) is generated at checkout, included in the order payload, saved in the Catalyst database, and rendered prominently on the student ticket.
- **Instant UPI Intent & Dynamic QR:** Integrates with UPI IDs (hkbkstudentpoint@upi) with automatic amount population, supporting GPay, PhonePe, and Paytm.
- **Cash Reservation Mode:** Students can select Cash at Counter, generating their pickup ticket immediately while locking the order in pending_cash state.
- **Live Ticket Screen:** Displays Order #ID, the 4-digit pickup code in 52pt font, quick copy button, and real-time status steps (Queued -> Printing -> Ready!).

5. Backend Architecture & Seller Integration Handbook

5.1 Zoho Catalyst Serverless Function & ZCQL Data Store

The backend runs as a Node 18 Advanced I/O function in functions/api/index.js. Orders are stored in the relational orders table via ZCQL queries:

Column	Type	Description
id	VarChar (PK)	UUID Order identifier (e.g. 34c882d4-f46e-4557...)
status	VarChar	Current order lifecycle state (pending_payment, processing, printing, ready, completed)
basket	LongText (JSON)	Full basket JSON containing files, pages, duplex settings, services, and pickupCode
file_names	LongText (JSON)	Array of uploaded file names for quick counter inspection
total	Double	Server-verified total price in INR (Rs)
quote_pending	Boolean	True if custom counter quote is required
created_at / updated_at	BigInt	Epoch timestamps for chronological sorting

5.2 API Endpoint Directory

- **POST /server/api/upload:** Uploads uncompressed raw file stream to Catalyst File Store folder print_files. Returns { cloudId, name, size }.
- **POST /server/api/orders:** Validates file parameters, recalculates exact math prices, stores pickupCode and basket, and returns { orderId, status, total, quotePending, pickupCode }.
- **GET /server/api/orders:** Returns live stream of orders ordered by created_at DESC. Parses basket to expose pickupCode and file names for seller counter dashboard.
- **GET /server/api/orders/:id:** Returns single order state and pickup code for the student ticket screen.
- **POST /server/api/orders/:id/pay:** Transitions order status from pending_payment to processing upon UPI payment confirmation.
- **POST /server/api/orders/:id/status:** Allows the seller counter dashboard to transition status to printing, ready, or completed.

5.3 Counter / Seller Ingestion Workflow

1. Student arrives at counter and shows 4-digit code (e.g. 4821) on their mobile ticket screen.
2. Counter app fetches GET /server/api/orders and filters by pickupCode == "4821".
3. Counter operator or automated agent downloads the original file using the file cloudId.
4. Job is dispatched to printer; operator clicks "Mark Ready" (POST /orders/:id/status { status: "ready" }).
5. Student phone immediately updates to "Ready! Show code 4821 at the counter".

Final System Verification Summary

Student Frontend: BUILT & DEPLOYED (Vite 8 bundle with code-split pptx renderer)
API Endpoints: DEPLOYED & LIVE on Zoho Catalyst serverless infrastructure
Document Parsers: Verified on PDF, PPTX (9+ slides), DOCX, and JPG/PNG formats
Pricing Engine: Authoritative match on client and server (Rs 1.50 / Rs 0.85 / Rs 8.00)
Next Step: Ready to connect seller side dashboard and counter agent end-to-end.